Advanced Counting Techniques

8

Introduction

- How many bit strings of length *n* do not contain two consecutive zeros. (recurrence relation)
- Many counting problems can be solved using formal power series, where the coefficients of powers of *x* represent terms of sequence we are interested in. (generating function)
- How many ways are there to assign seven jobs to three employees so that each employee is assigned at least one job. (principle of inclusion-exclusion)
- The techniques studied in this chapter, together with the basic techniques of Chapter 6, can be used to solve many counting problems.

Advanced Counting Techniques

8.1 Applications of Recurrence Relations
8.2 Solving Linear Recurrence Relations
8.3 Divide-and-Conquer Algorithms and Recurrence Relations
8.4 Generating Functions
8.5 Inclusion-Exclusion
8.6 Applications of Inclusion-Exclusion

8.1 Applications of Recurrence Relations

- Problem: The number of bacteria in a colony doubles every hour. If a colony begins with five bacteria, how many will be present in *n* hours?

  A variety of counting problems can be modeled using recurrence relations.

- Study a variety of counting problems that can be modeled using recurrence relations. Find explicit formulae of the terms of sequences that satisfy certain types of recurrence relations.

Recurrence Relations

- A *recurrence relation* for the sequence $\{a_n\}$ is an equation that expresses a_n in terms of one or more of the previous terms of the sequence, namely, $a_0, a_1, \ldots, a_{n-1}$, for all integers n with $n \geq n_0$, where n_0 is a nonnegative integer.

$$\boldsymbol{a_n = f(a_0, a_1, a_2, \ldots, a_{n-1}) \quad n \geq n_0}$$

- For example,
- The Fibonacci sequence $\boldsymbol{a_n = a_{n-1} + a_{n-2}}$.
- Pascal's recursion for the binomial coefficient is a two variable recurrence equation

$$\binom{n+1}{k} = \binom{n}{k} + \binom{n}{k-1}$$

Recurrence Relations

- A sequence is called a *solution* of a recurrence relation if its terms satisfy the recurrence relation.
- Let $\{a_n\}$ be a sequence that satisfies the recurrence relation $a_n = a_{n-1} - a_{n-2}$ for $n = 2, 3, 4,\ldots$ Suppose that $a_0 = 3$ and $a_1 = 5$. What are a_2 and a_3?

  $a_2 = a_1 - a_0 = 5-3 = 2$

  $a_3 = a_2 - a_1 = 2-5 = -3$

- Consider the recurrence relation:

  $a_n = 2a_{n-1} - a_{n-2}$ for $n = 2, 3, 4, \ldots$

  - Is the sequence $\{a_n\}$ defined by $a_n = 3n$ is a solution of this recurrence relation? √
  - Is the sequence $\{a_n\}$ defined by $a_n = 2^n$ is a solution of this recurrence relation? X
  - Is the sequence $\{a_n\}$ defined by $a_n = 5$ is a solution of this recurrence relation? √

Recurrence Relations vs. Recursive Definitions

- A *recurrence relation* is a type of *recursive definition* of the sequence.

- *Recursive definitions* can be used to solve counting problems. When they are used in this way, the rule for finding terms from those that precede them is called a *recurrence relation*.

Some notes

- Initial conditions: for a sequence specify the terms that precede the first term where the recurrence relation takes effect.
- The recurrence relation and initial conditions uniquely determine a sequence.
- Any term of the sequence can be found from the initial conditions using the recurrence relation a sufficient number of times. However, there are better ways for computing the terms of certain classes of sequences defined by recurrence relations and initial conditions.

Question?

- Give a recurrence relation, how many initial conditions are needed to uniquely identify the sequence?

- The *degree* of a recurrence relation

  $a_n = a_{n-1} + a_{n-8}$ ---- a recurrence relation of degree 8

- For example,
- In the Fibonacci recurrence $a_n = a_{n-1} + a_{n-2}$ we must specify a_0 and a_1 .
- In Pascal's identity

$$\binom{n+1}{k} = \binom{n}{k} + \binom{n}{k-1}$$

We must specify $C(1,0)$ and $C(1,1)$.

Modeling with recurrence relations

- Suppose that a person deposits $10,000 in a savings account at a bank yielding 11% per year with interest compounded annually. How much will be in the account after 30 years?

$$P_n=P_{n-1}+0.11P_{n-1}=(1.11)P_{n-1}$$
$$P_{30}=(1.11)^{30}10000=22892297$$

- A young pair of rabbits (one of each sex) is placed on an island. A pair of rabbits does not breed until they are 2 months old. After they are 2 months old, each pair produces another pair each month. Find a recurrence relation for the number of pairs of rabbits on the island after n months, assuming that no rabbits ever die.

$$f_n = f_{n-1} + f_{n-2} \text{ for } n > 2$$
$$f_1 = 1,\ f_2 = 1$$

Example

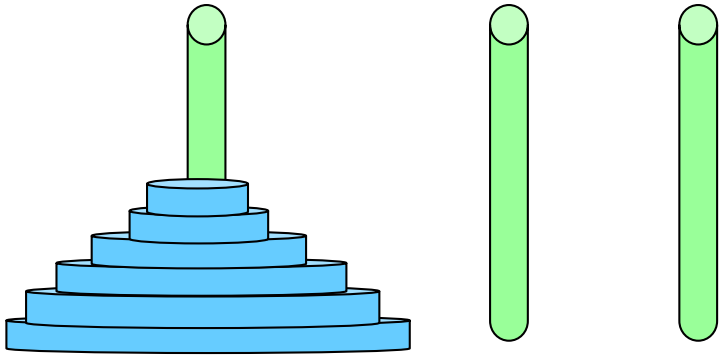
- The Tower of Hanoi

  The famous puzzle of the late nineteenth century

  $H_n=2H_{n-1}+1$

  $H_1=1$

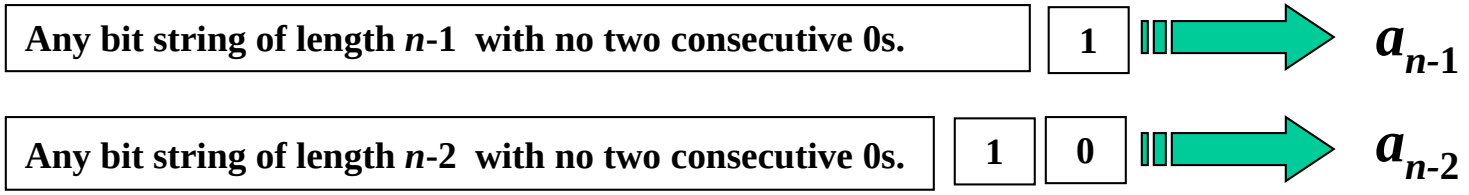
  H_n denote the number of moves needed to solve The Tower of Hanoi problem with n disks



$$
\begin{aligned}
H_n &= 2H_{n-1}+1 \\
&= 2(2H_{n-2}+1)+1 \\
&= 2^2H_{n-2}+2+1 \\
&= 2^2(2H_{n-3}+1)+2+1 \\
&= 2^3H_{n-3}+2^2+2+1 \\
&= \dots \\
&= 2^{n-1}H_1+2^{n-2}+\dots+2+1 \\
&= 2^{n-1}+2^{n-2}+\dots+2+1 \\
&= 2^n-1
\end{aligned}
$$

Example

- Find a recurrence relation and give initial conditions for the number of bit strings of length *n* that do not have two consecutive 0s.
- Let a_n denote the number of bit strings of length *n* that don't have two consecutive 0s.



$a_n = a_{n-1} + a_{n-2},$ $a_1=2,$ $a_2=3$

Example

- Codeword Enumeration: A computer system considers a string of decimal digits a valid codeword if it contains an even number of 0 digits. Let a_n be the number of valid n-digit codeword. Find a recurrence relation for a_n.

$$a_n = 9a_{n-1} + (10^{n-1} - a_{n-1})$$

Example

- Codeword Enumeration: A computer system considers a string of decimal digits a valid codeword if it contains an even number of 0 digits. Let a_n be the number of valid n-digit codeword. Find a recurrence relation for a_n.

$$a_n = 9a_{n-1} + (10^{n-1} - a_{n-1})$$

Example

- Find a recurrence relation for C_n, the number of ways to parenthesize the product of $n + 1$ numbers, $x_0 \cdot x_1 \cdot x_2 \cdot \cdots \cdot x_n$, to specify the order of multiplication. For example, $C_3 = 5$, since all the possible ways to parenthesize 4 numbers are
$$((x_0 \cdot x_1) \cdot x_2) x_3, (x_0 . (x_1 \cdot x_2)) \cdot x_3, (x_1 \cdot x_2) \cdot (x_2 \cdot x_3), x_0 \cdot ((x_1 \cdot x_2) \cdot x_3), x_0 \cdot (x_1 \cdot (x_2 \cdot x_3))$$

- Solution: Note that however parentheses are inserted in $x_0 \cdot x_1 \cdot x_2 \cdot \cdots \cdot x_n$, one "·" operator remains outside all parentheses. This final operator appears between two of the $n + 1$ numbers, say x_k and x_{k+1}. Since there are C_k ways to insert parentheses in the product $x_0 \cdot x_1 \cdot x_2 \cdot \cdots \cdot x_k$ and C_{n-k-1} ways to insert parentheses in the product $x_{k+1} \cdot x_{k+2} \cdot \cdots \cdot x_n$, we have

$$C_n = C_0 C_{n-1} + C_1 C_{n-2} + \cdots + C_{n-2} C_1 + C_{n-1} C_0$$

$$= \sum_{k=0}^{n-1} C_k C_{n-k-1}$$

  - The initial conditions are $C_0 = 1$ and $C_1 = 1$.

- The sequence $\{C_n\}$ is the sequence of Catalan Numbers

Dynamic Programming

- A DP is an algorithmic technique which is usually based on a recurrent relation and one (or some) starting states. A sub-solution of the problem is constructed from previously found ones. DP solutions have a polynomial complexity which assures a much faster running time than other techniques like backtracking, brute-force etc.
- Example 6 in Page 534-536 (talk schedule)

1D - Dynamic Programming

给定一个整数数组 nums，找到其中最长严格递增子序列的长度。

示例

输入：nums = [10, 9, 2, 5, 3, 7, 101, 18]
输出：最长递增子序列是 [2, 3, 7, 101]（或者 [2, 5, 7, 18]），长度为 4

动态规划思路

定义状态: dp[i] 表示以 nums[i] 结尾的最长递增子序列的长度。

状态转移：对于每个 i，检查所有 j < i：

如果 nums[j] < nums[i]，说明可以把 nums[i] 接在以 nums[j] 结尾的序列后面，形成更长的序列。

因此：dp[i] = max(dp[i], dp[j] + 1)

初始化

每个元素自身可以作为一个长度为 1 的子序列，以 dp[i] = 1。

答案

max(dp[0], dp[1], ..., dp[n-1])

Dynamic Programming

Python

```
def lengthOfLIS(nums):
    if not nums:
        return 0
    n = len(nums)
    dp = [1] * n
    for i in range(n):
        for j in range(i):
            if nums[j] < nums[i]:
                dp[i] = max(dp[i], dp[j] + 1)
    return max(dp)

# 测试
nums = [10, 9, 2, 5, 3, 7, 101, 18]
print(lengthOfLIS(nums))  # 输出 4
```

时间复杂度：$O(n^2)$
n是数组长度

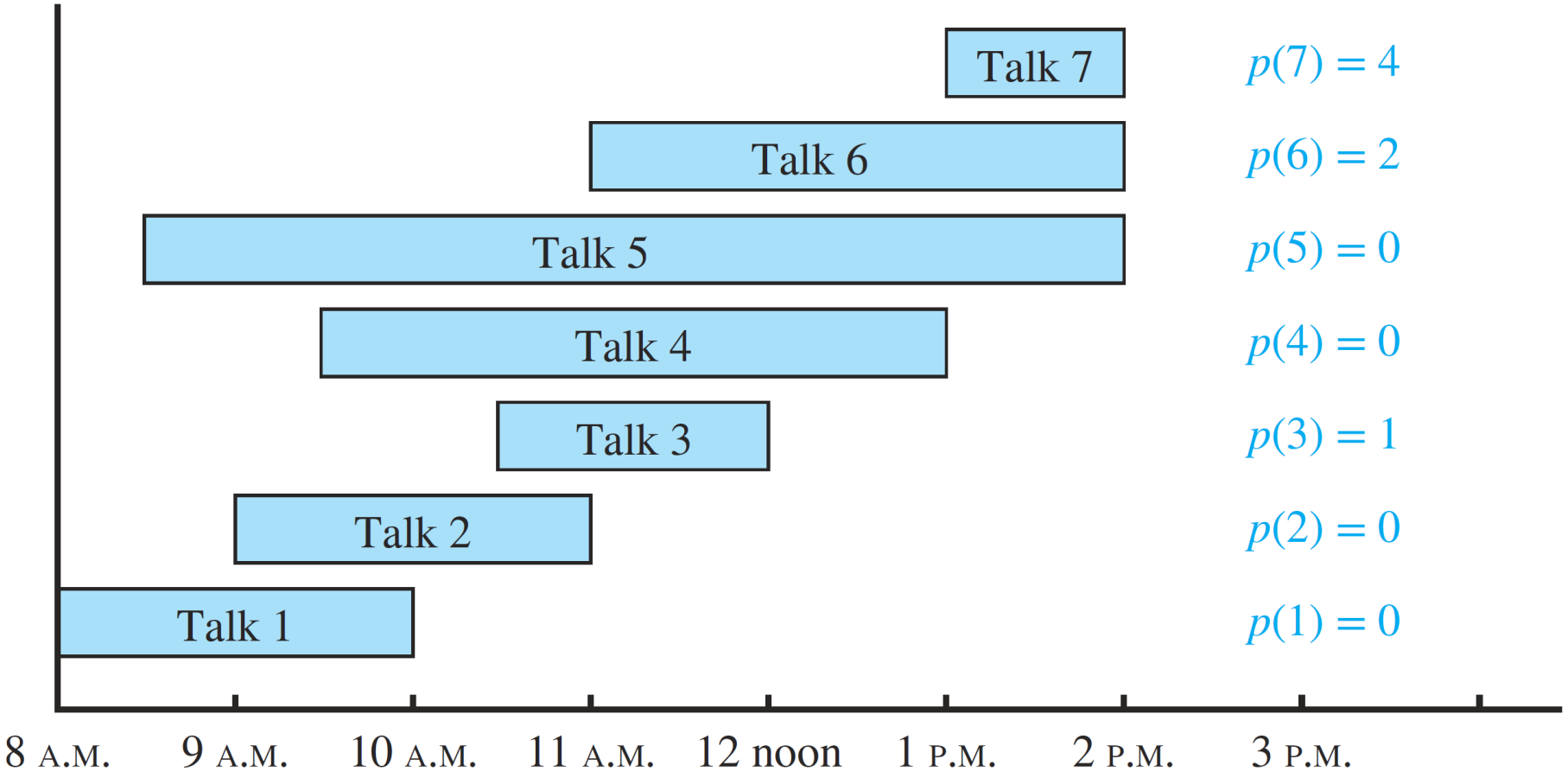
如何优化到O(nlogn)

Dynamic Programming

We formalize this problem by supposing that we have n talks, where talk j begins at time t_j, ends at time e_j, and will be attended by w_j students. We want a schedule that maximizes the total number of student attendees. That is, we wish to schedule a subset of talks to maximize the sum of w_j over all scheduled talks. (Note that when a student attends more than one talk, this student is counted according to the number of talks attended.) We denote by $T(j)$ the maximum number of total attendees for an optimal schedule from the first j talks, so $T(n)$ is the maximal number of total attendees for an optimal schedule for all n talks.

We first sort the talks in order of increasing end time. After doing this, we renumber the talks so that $e_1 \leq e_2 \leq \cdots \leq e_n$. We say that two talks are **compatible** if they can be part of the same schedule, that is, if the times they are scheduled do not overlap (other than the possibility one ends and the other starts at the same time). We define $p(j)$ to be largest integer i, $i < j$, for which $e_i \leq s_j$, if such an integer exists, and $p(j) = 0$ otherwise. That is, talk $p(j)$ is the talk ending latest among talks compatible with talk j that end before talk j ends, if such a talk exists, and $p(j) = 0$ if there are no such talks.

Dynamic Programming



To develop a dynamic programming algorithm for this problem, we first develop a key recurrence relation. To do this, first note that if $j \leq n$, there are two possibilities for an optimal schedule of the first j talks (recall that we are assuming that the n talks are ordered by increasing end time): *(i)* talk j belongs to the optimal schedule or *(ii)* it does not.

Case (i): We know that talks $p(j) + 1, \ldots, j - 1$ do not belong to this schedule, for none of these other talks are compatible with talk j. Furthermore, the other talks in this optimal schedule must comprise an optimal schedule for talks $1, 2, \ldots, p(j)$. For if there were a better schedule for talks

$1, 2, \ldots, p(j)$, by adding talk j, we will have a schedule better than the overall optimal schedule. Consequently, in case *(i)*, we have $T(j) = w_j + T(p(j))$.

Case (ii): When talk j does not belong to an optimal schedule, it follows that an optimal schedule from talks $1, 2, \ldots, j$ is the same as an optimal schedule from talks $1, 2, \ldots, j - 1$. Consequently, in case *(ii)*, we have $T(j) = T(j - 1)$. Combining cases *(i)* and *(ii)* leads us to the recurrence relation

$$T(j) = \max(w_j + T(p(j)), T(j - 1)).$$

Now that we have developed this recurrence relation, we can construct an efficient algorithm, Algorithm 1, for computing the maximum total number of attendees. We ensure that the algorithm is efficient by storing the value of each $T(j)$ after we compute it. This allows us to compute $T(j)$ only once. If we did not do this, the algorithm would have exponential worst-case complexity. The process of storing the values as each is computed is known as **memoization** and is an important technique for making recursive algorithms efficient.

Dynamic Programming

ALGORITHM 1 Dynamic Programming Algorithm for Scheduling Talks.

procedure *Maximum Attendees* ($s_1, s_2, \ldots, s_n$: start times of talks;
$e_1, e_2, \ldots, e_n$: end times of talks; $w_1, w_2, \ldots, w_n$: number of attendees to talks)
sort talks by end time and relabel so that $e_1 \leq e_2 \leq \cdots \leq e_n$
for $j := 1$ **to** n
    if no job i with $i < j$ is compatible with job j
        $p(j) = 0$
    else $p(j) := \max\{i \text{ — } i < j \text{ and job } i \text{ is compatible with job } j\}$
$T(0) := 0$
for $j := 1$ **to** n
    $T(j) := \max(w_j + T(p(j)), T(j-1))$
return $T(n)\{T(n)$ is the maximum number of attendees$\}$

2D - Dynamic Programming

给定两个字符串 text1 和 text2，返回它们的最长公共子序列的长度

子序列：不要求连续，但顺序必须相对一致。

示例

输入：text1 = "abcde", text2 = "ace"
输出：最长公共子序列是 "ace"，长度为 3。

输入：text1 = "abc", text2 = "def"
输出：0

状态定义

设 dp[i][j] 表示：text1 的前 i 个字符（即 text1[0..i-1]）和 text2 的前 j 个字符（即 text2[0..j-1]）的最长公共子序列的长度

最终答案是 dp[len(text1)][len(text2)]

状态转移

if text1[i-1] == text2[j-1]: dp[i][j] = dp[i-1][j-1] + 1

else: dp[i][j] = max(dp[i-1][j], dp[i][j-1])

Dynamic Programming

```
def longestCommonSubsequence(text1: str, text2: str) -> int:
    m, n = len(text1), len(text2)
    # 创建 DP 表，多一行一列用于处理空字符串
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    # 填充 DP 表
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if text1[i-1] == text2[j-1]:
                dp[i][j] = dp[i-1][j-1] + 1
            else:
                dp[i][j] = max(dp[i-1][j], dp[i][j-1])
    return dp[m][n]
# 测试
print(longestCommonSubsequence("abcde", "ace"))   # 3
print(longestCommonSubsequence("abc", "def"))     # 0
print(longestCommonSubsequence("abcd", "acbd"))   # 3 ("abd" 或 "acd")
```

时间复杂度：O(m×n)

Exercises

- **P.536: 10, 24, 25, 53, 56**

8.2 Solving Linear Recurrence Relations

- The Methods of Solving Recurrence Relations
  - Iterative approach
  - Generating functions
  - Use a systematic way to solve an important class of recurrence relations

- Solving recurrence relations can be very difficult unless the recurrence equation has a special form.

Solving Linear Recurrence Relations

- A wide variety of recurrence relations occur in models. One important class of recurrence relation (linear combinations of previous terms) can be explicitly solved in a systematic way.
- DEFINITION: A linear homogeneous recurrence relation of degree k with constant coefficients is a recurrence relation of the form

$$a_n = c_1 a_{n-1} + c_2 a_{n-2} + \ldots + c_k a_{n-k}$$

where $c_1, c_2, \ldots, c_k$ are real numbers and $c_k \neq 0$

- A sequence satisfying the recurrence relation in the definition is uniquely determined by the recurrence relation and the k initial conditions

$$a_0 = C_0, a_1 = C_1, \ldots, a_{k-1} = C_{k-1}$$

Some Terminologies

- *linear*

  - linear combination of previous terms

  - no products of the a_i' s

- *constant coefficients*

  the coefficients in the sum of the a_i' s are constants,

  independent of *n*.

- *degree k*

  a_n is a function of only the previous *k* terms in the sequence

- *homogeneous*

  If we put all the *ai*' s on the left side of the equation and everything else on the right side, then the right side is 0. Otherwise *inhomogeneous* or *nonhomogeneous*.

Examples

- What is the degree of $a_n = 2a_{n-1} + a_{n-2}$?
- What is the degree of $a_n = a_{n-2} + 3a_{n-3}$?
- What is the degree of $a_n = 3a_{n-4}$?
- $a_n = a_{n-1} + a^2_{n-2}$ is not linear.
- $H_n = 2H_{n-1} + 1$ is not homogeneous
- $B_n = nB_{n-1}$ does not have constant coefficients.

Examples

(1) $a_n = (1.02)a_{n-1}$

linear; constant coefficients; homogeneous; degree 1

(2) $a_n = (1.02)\, a_{n-1} + 2^{n-1}$

linear; constant coefficients; nonhomogeneous; degree 1

(3) $a_n = a_{n-1} + a_{n-2} + a_{n-3} + 2^{n-3}$

linear; constant coefficients; nonhomogeneous; degree 3

(4) $a_n = n\, a_{n-1} + n^2\, a_{n-2} + a_{n-1}\, a_{n-2}$

nonlinear; coefficients are not constants; homogeneous; degree 2

Solving linear homogeneous recurrence relations with constant coefficients

$$a_n = c_1 a_{n-1} + c_2 a_{n-2} + \cdots + c_k a_{n-k}$$

- Assume the form of solution is $a_n = r^n$, then

  $a_n = c_1 a_{n-1} + c_2 a_{n-2} + \ldots + c_k a_{n-k}$

  $r^n = c_1 r^{n-1} + c_2 r^{n-2} + \ldots + c_k r^{n-k}$

  $r^k - c_1 r^{k-1} - c_2 r^{k-1} - \ldots - c_{k-1} r - c_k = 0$

- The sequence $\{a_n\}$ with $a_n = r^n$ is a solution iff r is a solution of the last equation, which is called the characteristic equation of the recurrence relation. The solutions of this equation are called the characteristic roots of the recurrence relation.

Solving 1st Order LHRRCC

What is the solution of the recurrence relation

$$a_{n+2} = 3a_{n+1}, a_0 = 4$$

Solution:

(1) The Characteristic equation of the recurrence relation is $r - 3 = 0$.

(2) Find the root of the characteristic equation: $r_1 = 3$

(3) Compute the general solution: $a_n = c3^n$

(4) Find the constants based on the initial conditions:

$a_0 = c3^n = 4$

(5) Produce the specific solution: $a_n = 4 \cdot 3^n$

Solving 2nd Order LHRRCC

- Problem: $a_n = c_1 a_{n-1} + c_2 a_{n-2}$ with some constant values for a_0 and a_1, and assume that the solution is $a_n = r^n$, where r is a constant and $r \neq 0$. Give the solution of a_n .

Step 1

- Solve the characteristic quadratic equation $r^2 - c_1 r - c_2 = 0$ to find the characteristic roots r_1 and r_2

$$r_{1,2} = \frac{c_1 \pm \sqrt{c_1^{\,2} - 4c_2}}{2}$$

- Case I: The roots are not equal

$$a_n = \alpha_1 r_1^{\,n} + \alpha_2 r_2^{\,n}$$

- Case II: The roots are equal ($r_1 = r_2 = r_0$)

$$a_n = \alpha_1 r_0^{\,n} + \alpha_2 n r_0^{\,n}$$

Step 2

- Apply the initial conditions to the equations derived in the previous step.
  - Case I: The roots are not equal

$$a_0 = \alpha_1 r_1^0 + \alpha_2 r_2^0 = \alpha_1 + \alpha_2$$

$$a_1 = \alpha_1 r_1^1 + \alpha_2 r_2^1 = \alpha_1 r_1 + \alpha_2 r_2$$

  - Case II: The roots are equal

$$a_0 = \alpha_1 r_0^0 + \alpha_2 \cdot 0 \cdot r_0^0 = \alpha_1$$

$$a_1 = \alpha_1 r_0^1 + \alpha_2 \cdot 1 \cdot r_0^1 = (\alpha_1 + \alpha_2) r_0$$

- Solve the appropriate pair of equations for α_1 and α_2.

Step 3

- Substitute the values of α_1, α_2, and the root(s) into the appropriate equation in step 1 to find the explicit formula for a_n.

  Case I: The roots are not equal

  $$a_n = \alpha_1 r_1^n + \alpha_2 r_2^n$$

  Case II: The roots are equal ($r_1 = r_2 = r_0$)

  $$a_n = \alpha_1 r_0^n + \alpha_2 n r_0^n$$

Theorem 1 and 2

【 Theorem 1】 Let c_1, c_2 be real numbers. Suppose that $r^2 - c_1 r - c_2 = 0$ has two distinct roots r_1, r_2 . Then the Sequence $\{a_n\}$ is a solution of the recurrence relation $a_n = c_1 a_{n-1} + c_2 a_{n-2}$ if and only if $a_n = \alpha_1 r_1^n + \alpha_2 r_2^n$ for $n = 0,1,2,\cdots$, where α_1, α_2 are constants.

【 Theorem 2】 Let c_1, c_2 be real numbers with $c_2 \neq 0$. Suppose that $r^2 - c_1 r - c_2 = 0$ has only one root r_0 . A sequence $\{a_n\}$ is a solution of the recurrence relation $a_n = c_1 a_{n-1} + c_2 a_{n-2}$ if and only if $a_n = \alpha_1 r_0^n + \alpha_2 n r_0^n$ for $n = 0,1,2,\cdots$, where α_1, α_2 are constants.

Example: Find an explicit formula for the Fibonacci numbers.

Solution:

$$f_n = f_{n-1} + f_{n-2}, \quad f_0 = 0, \quad f_1 = 1$$

(1) Determine the characteristic equation: $r^2 - r - 1 = 0$

(2) Find its roots: $r_1 = \frac{1+\sqrt{5}}{2}, r_2 = \frac{1-\sqrt{5}}{2}$

(3) Compute the general solution:

$$f_n = \alpha_1 (\frac{1+\sqrt{5}}{2})^n + \alpha_2 (\frac{1-\sqrt{5}}{2})^n, \alpha_1, \alpha_2 \text{ are constant.}$$

(4) Determine α_1, α_2 **:** $\alpha_1 = \frac{1}{\sqrt{5}}, \alpha_2 = -\frac{1}{\sqrt{5}}$

Consequently, the fibonacci numbers are given by

$$f_n = \frac{1}{\sqrt{5}} (\frac{1+\sqrt{5}}{2})^n - \frac{1}{\sqrt{5}} (\frac{1-\sqrt{5}}{2})^n$$

Example: $a_n = 6a_{n-1} - 9a_{n-2}, a_0 = a_1 = 1$

Solution:

(1) Recurrence system : $a_n - 6a_{n-1} + 9a_{n-2} = 0$

(2) Determine the characteristic equation: $(b-3)^2 = 0$

(3) Find its roots : $b_1 = b_2 = 3$

(4) Compute the general solution: $a_n = (\alpha_1 + \alpha_2 n)3^n$

(5) Solve for coefficients :

$$\begin{cases} a_0 = 1 = \alpha_1 \\ a_1 = 1 = (\alpha_1 + \alpha_2) \cdot 3 \end{cases}$$

Consequently, $a_n = (1 - \frac{2}{3}n)3^n$

Solving k^{nd} Order LHRRCC

Theorem 3:

Let $c_1, c_2, \cdots, c_k$ be real numbers. Suppose that the characteristic equation

$$r^k - c_1 r^{k-1} - \cdots - c_k = 0$$

has k distinct roots $r_1, r_2, \cdots, r_k$. Then a sequence $\{a_n\}$ is a solution of the recurrence relation

$$a_n = c_1 a_{n-1} + c_2 a_{n-2} + \cdots + c_k a_{n-k}$$

If and only if

$$a_n = \alpha_1 r_1^n + \alpha_2 r_2^n + \cdots + \alpha_k r_k^n$$

for $n = 0,1,2,\cdots$, where $\alpha_1, \alpha_2, \cdots, \alpha_k$ are constants.

Continue

Theorem 4

Let $c_1, c_2, \cdots, c_k$ be real numbers. Suppose that the characteristic equation

$$r^k - c_1 r^{k-1} - \cdots - c_k = 0$$

has t distinct roots $r_1, r_2, \cdots, r_t$ with multiplicities $m_1, m_2, \cdots, m_t$, respectively, so that $m_i \geq 1$ for $i = 1,2,\cdots,t$ and $m_1 + m_2 + \cdots + m_t = k$.

Then a sequence $\{a_n\}$ is a solution of the recurrence relation

$$a_n = c_1 a_{n-1} + c_2 a_{n-2} + \cdots + c_k a_{n-k}$$

If and only if

$$\begin{aligned} a_n = {} & (\alpha_{1,0} + \alpha_{1,1} n + \cdots + \alpha_{1,m_1-1} n^{m_1-1}) r_1^n \\ & + (\alpha_{2,0} + \alpha_{2,1} n + \cdots + \alpha_{2,m_2-1} n^{m_2-1}) r_2^n \\ & + \cdots + (\alpha_{t,0} + \alpha_{t,1} n + \cdots + \alpha_{1,m_{t1}-1} n^{m_{t1}-1}) r_t^n \end{aligned}$$

for $n = 0,1,2,\cdots$, where $\alpha_{i,j}$ are constants for $1 \leq i \leq t$ and $0 \leq j \leq m_i - 1$.

Examples

- Page 545-547: example 6,7,8

EXAMPLE 7 Suppose that the roots of the characteristic equation of a linear homogeneous recurrence relation are 2, 2, 2, 5, 5, and 9 (that is, there are three roots, the root 2 with multiplicity three, the root 5 with multiplicity two, and the root 9 with multiplicity one). What is the form of the general solution?

Solution: By Theorem 4, the general form of the solution is

$$(\alpha_{1,0} + \alpha_{1,1}n + \alpha_{1,2}n^2)2^n + (\alpha_{2,0} + \alpha_{2,1}n)5^n + \alpha_{3,0}9^n.$$

Examples

EXAMPLE 8 Find the solution to the recurrence relation

$$a_n = -3a_{n-1} - 3a_{n-2} - a_{n-3}$$

with initial conditions $a_0 = 1$, $a_1 = -2$, and $a_2 = -1$.

Solution: The characteristic equation of this recurrence relation is

$$r^3 + 3r^2 + 3r + 1 = 0.$$

Because $r^3 + 3r^2 + 3r + 1 = (r + 1)^3$, there is a single root $r = -1$ of multiplicity three of the characteristic equation. By Theorem 4 the solutions of this recurrence relation are of the form

To find the constants $\alpha_{1,0}$, $\alpha_{1,1}$, and $\alpha_{1,2}$, use the initial conditions. This gives

$$a_0 = 1 = \alpha_{1,0},$$
$$a_1 = -2 = -\alpha_{1,0} - \alpha_{1,1} - \alpha_{1,2},$$
$$a_2 = -1 = \alpha_{1,0} + 2\alpha_{1,1} + 4\alpha_{1,2}.$$

The simultaneous solution of these three equations is $\alpha_{1,0} = 1$, $\alpha_{1,1} = 3$, and $\alpha_{1,2} = -2$. Hence, the unique solution to this recurrence relation and the given initial conditions is the sequence $\{a_n\}$ with

$$a_n = (1 + 3n - 2n^2)(-1)^n.$$

Linear nonhomogeneous recurrence relations with constant coefficients

- A linear nonhomogeneous recurrence relation with constant coefficients is a recurrence relation of the form

$$a_n = c_1 a_{n-1} + c_2 a_{n-2} + \ldots + c_k a_{n-k} + F(n)$$

where $c_1, c_2, \ldots, c_k$ are real numbers, and $F(n)$ is a function not identically zero depending only on n. The recurrence relation

$$a_n = c_1 a_{n-1} + c_2 a_{n-2} + \ldots + c_k a_{n-k}$$

is called the associated homogeneous recurrence relation.

Examples

- $a_n=a_{n-1}+2^n$
- $a_n=a_{n-1}+a_{n-2}+n^2+n+1$
- $a_n=3a_{n-1}+n3^n$
- $a_n=a_{n-1}+a_{n-2}+a_{n-3}+n!$

Theorem 5

- IF $\{a_n^{(p)}\}$ is particular solution of the linear nonhomogeneous recurrence relation with constant coefficients

$$a_n = c_1 a_{n-1} + c_2 a_{n-2} + \ldots + c_k a_{n-k} + F(n)$$

then every solution is of the form $\{a_n^{(p)} + a_n^{(h)}\}$, where $a_n^{(h)}$ is a solution of the associated homogeneous recurrence relation

$$a_n = c_1 a_{n-1} + c_2 a_{n-2} + \ldots + c_k a_{n-k}$$

- The key to solving nonhomogeneous recurrence relations with constant coefficients is finding a particular solution.

Proof

Proof: Because $\{a_n^{(p)}\}$ is a particular solution of the nonhomogeneous recurrence relation, we know that

$$a_n^{(p)} = c_1 a_{n-1}^{(p)} + c_2 a_{n-2}^{(p)} + \cdots + c_k a_{n-k}^{(p)} + F(n).$$

Now suppose that $\{b_n\}$ is a second solution of the nonhomogeneous recurrence relation, so that

$$b_n = c_1 b_{n-1} + c_2 b_{n-2} + \cdots + c_k b_{n-k} + F(n).$$

Subtracting the first of these two equations from the second shows that

$$b_n - a_n^{(p)} = c_1(b_{n-1} - a_{n-1}^{(p)}) + c_2(b_{n-2} - a_{n-2}^{(p)}) + \cdots + c_k(b_{n-k} - a_{n-k}^{(p)}).$$

It follows that $\{b_n - a_n^p\}$ is a solution of the associated homogeneous linear recurrence, say, $\{a_n^{(h)}\}$. Consequently, $b_n = a_n^{(p)} + a_n^{(h)}$ for all n. ◁

Theorem 6

【 Theorem 6】 Assume a linear nonhomogeneous recurrence equation with constant coefficients with the nonlinear part *F(n)* of the form

$$F(n) = (b_t n^t + b_{t-1} n^{t-1} + \cdots + b_1 n + b_0) s^n$$

If *s* is not a root of the characteristic equation of the associated homogeneous recurrence equation, there is a particular solution of the form

$$(p_t n^t + p_{t-1} n^{t-1} + \cdots + p_1 n + p_0) s^n$$

If *s* is a root of multiplicity *m* , a particular solutions is of the form

$$n^m (p_t n^t + p_{t-1} n^{t-1} + \cdots + p_1 n + p_0) s^n$$

Example:

$$a_n = 6a_{n-1} - 9a_{n-2} + F(n),$$

$$where F(n) = 3^n, n3^n, n^2 2^n, and (n^2+1)\cdot 3^n$$

Solution:

(1) The general solution of the associated homogeneous recurrence equation :

$$a_n = (\alpha_1 + \alpha_2 n)\cdot 3^n$$

(2) A particular solution of the form:

$$p_0 n^2 3^n \; if F(n) = 3^n$$

$$n^2(p_1 n + p_0)\cdot 3^n \; if F(n) = n3^n$$

$$(p_2 n^2 + p_1 n + p_0)\cdot 2^n \; if F(n) = n^2 2^n$$

$$n^2(p_2 n^2 + p_1 n + p_0)\cdot 3^n \; if F(n) = (n^2+1)3^n$$

Example: **Let a_n be the sum of the first n positive integers. Note that a_n satisfies the recurrence relation $a_n = a_{n-1} + n$. Find The explicit formula of a_n**

Solution:

(1) The general solution of the associated homogeneous recurrence equation :

$$a_n^{(h)} = c \cdot (1)^n = c$$

(2) A particular solution of the form:

$$n(p_1 n + p_0) = p_1 n^2 + p_0 n$$

(3) Find p_1,p_0:

$$p_1 n^2 + p_0 n = p_1 (n-1)^2 + p_0 (n-1) + n$$

$$n(2p_1 - 1) + (p_0 - p_1) = 0$$

$$p_0 = p_1 = 1/2,$$

(4) Find c: using initial condition a_1=1

Example

Example Find all solutions of the recurrence relation $a_n = 3a_{n-1} + 2n$. What is the solution with $a_1 = 3$?

Solution To solve this linear nonhomogeneous recurrence relation with constant coefficients, we need to solve its associated linear homogeneous recurrence relation and to find a particular solution.

The associated linear homogeneous recurrence relation $a_n = 3a_{n-1}$. Its solution are $a_n^{(h)} = \alpha 3^n$, where α is a constant.

We now find a particular solution. Since $F(n) = 2n$ is a polynomial in n of degree one, one reasonable trial solution is a linear function in n, say, $p_n = cn + d$, where c and d are constants. Suppose that $p_n = cn + d$ is such a solution. Then the equation $a_n = 3a_n - 1 + 2n$ becomes $cn + d = 3(c(n-1) + d) + 2n$. This shows that $cn + d$ is a solution if and only if $c = -1$ and $d = -3/2$.

By theorem, all solution are of the form

$$a_n = a_n^{(p)} + a_n^{(h)} = -n - \frac{3}{2} + \alpha \cdot 3^n$$

Let $n = 1$ and $a_1 = 3$, we obtain $\alpha = 11/6$. The solution we seek is

$$a_n = -n - \frac{3}{2} + \frac{11}{6}3^n.$$

Example

Example Find all solutions of the recurrence relation

$$a_n = 5a_{n-1} - 6a_{n-2} + 7^n.$$

(1) The solution of its associated homogeneous recurrence relation are $a_n^{(h)} = \alpha_1 \cdot 3^n - \alpha_2 \cdot 2^n$.
(2) Since $F(n) = 7^n$, a reasonable trial solution is $a_n^{(p)} = C \cdot 7^n$. Substituting the terms of this sequence into recurrence relation implies that $C = 49/20$. Hence, $a_n^{(p)} = (49/20) \cdot 7^n$.
By Theorem all solutions are of the form

$$a_n = \alpha_1 \cdot 3^n - \alpha_2 \cdot 2^n + (49/20)7^n.$$

Exercises

- **P550: 20, 30, 40, 35, 52**

8.3 Divide-and-conquer algorithms and recurrence relations

- Many recursive algorithms take a problem with a given input and divide it into one or more smaller problems.
- Divide-and-conquer algorithms divide a problem into one or more instances of the same problem of small size and they conquer the problem by using the solutions of the smaller problems to find a solution of the original problem.
- Recurrence relations can be used to analyze the computational complexity of divide-and-conquer algorithm.

“分而治之”VS“动态规划”

Divide-and-Conquer Recurrence Relations

- **Divide-and-Conquer Recurrence Relation:**

  $f(n)=af(n/b)+g(n)$

  f(n) represents the number of operations required to solve the problem of size of n.

  g(n) is the number of operations required to combine the solutions of the subproblems into a solution of the original problem.

- **Example: Binary Search**

  $f(n)=f(n/2)+2$ when n is even

  Two comparisons are needed to implement this reduction.

  One is to determine which half of the list to use.

  The other is to determine whether any term of the list remain.

Example

- **Merge sort**

$$M(n)=2M(n/2)+n$$

- **Fast multiplication of integers**

$$f(2n)=3f(n)+Cn$$

Fast multiplication of integers

- **An algorithm for the fast multiplication of two $2n$-bit integers (assuming n is even) first splits each of the $2n$-bit integers into two blocks, each of n bits.**
- **Suppose that a and b are integers with binary expansions of length $2n$. Let**
- **$a = (a_{2n-1}a_{2n-2} \ldots a_1a_0)_2$ and $b = (b_{2n-1}b_{2n-2} \ldots b_1b_0)_2$.**
- **Let $a = 2^nA_1 + A_0$, $b = 2^nB_1 + B_0$, where**
- **$A_1 = (a_{2n-1} \ldots a_{n+1}a_n)_2$, $A_0 = (a_{n-1} \ldots a_1a_0)_2$,**
- **$B_1 = (b_{2n-1} \ldots b_{n+1}b_n)_2$, $B_0 = (b_{n-1} \ldots b_1b_0)_2$.**
- **The algorithm is based on the fact that ab can be rewritten as:**
- **$ab = (2^{2n} + 2^n)A_1B_1 + 2^n(A_1 - A_0)(B_0 - B_1) + (2^n + 1)A_0B_0$.**
- **This identity shows that the multiplication of two $2n$-bit integers can be carried out using three multiplications of n-bit integers, together with additions, subtractions, and shifts.**
- **Hence, if $f(n)$ is the total number of operations needed to multiply two n-bit integers, then**

$$f(2n) = 3f(n) + Cn$$

Estimating the Size of Divide-and-Conquer Functions

Theorem 1: Let *f* be an increasing function that satisfies the recurrence relation

$$f(n) = af(n/b) + c$$

whenever *n* is divisible by *b*, where $a \geq 1$, *b* is an integer greater than 1, and *c* is a positive real number. Then

$$f(n) \text{ is } \begin{cases} O(n^{\log_b a}) & \text{if } a > 1 \\ O(\log n) & \text{if } a = 1. \end{cases}$$

Furthermore, when $n = b^k$ and $a \neq 1$, where *k* is a positive integer,

$$f(n) = C_1 n^{\log_b a} + C_2$$

where $C_1 = f(1) + c/(a-1)$ and $C_2 = -c/(a-1)$.

Proof: page 556-557

Estimating the Size of Divide-and-Conquer Functions

Proof: First let $n = b^k$. From the expression for $f(n)$ obtained in the discussion preceding the theorem, with $g(n) = c$, we have

$$f(n) = a^k f(1) + \sum_{j=0}^{k-1} a^j c = a^k f(1) + c \sum_{j=0}^{k-1} a^j.$$

When $a = 1$ we have

$$f(n) = f(1) + ck.$$

Because $n = b^k$, we have $k = \log_b n$. Hence,

$$f(n) = f(1) + c \log_b n.$$

$$\begin{aligned} f(n) &= af(n/b) + g(n) \\ &= a^2 f(n/b^2) + ag(n/b) + g(n) \\ &= a^3 f(n/b^3) + a^2 g(n/b^2) + ag(n/b) + g(n) \\ &\vdots \\ &= a^k f(n/b^k) + \sum_{j=0}^{k-1} a^j g(n/b^j). \end{aligned}$$

Because $n/b^k = 1$, it follows that

$$f(n) = a^k f(1) + \sum_{j=0}^{k-1} a^j g(n/b^j).$$

When n is not a power of b, we have $b^k < n < b^{k+1}$, for a positive integer k. Because f is increasing, it follows that $f(n) \le f(b^{k+1}) = f(1) + c(k+1) = (f(1) + c) + ck \le (f(1) + c) + c \log_b n$. Therefore, in both cases, $f(n)$ is $O(\log n)$ when $a = 1$.

Now suppose that $a > 1$. First assume that $n = b^k$, where k is a positive integer. From the formula for the sum of terms of a geometric progression (Theorem 1 in Section 2.4), it follows that

$$\begin{aligned} f(n) &= a^k f(1) + c(a^k - 1)/(a - 1) \\ &= a^k[f(1) + c/(a - 1)] - c/(a - 1) \\ &= C_1 n^{\log_b a} + C_2, \end{aligned}$$

because $a^k = a^{\log_b n} = n^{\log_b a}$ (see Exercise 4 in Appendix 2), where $C_1 = f(1) + c/(a - 1)$ and $C_2 = -c/(a - 1)$.

Now suppose that n is not a power of b. Then $b^k < n < b^{k+1}$, where k is a nonnegative integer. Because f is increasing,

$$\begin{aligned} f(n) &\le f(b^{k+1}) = C_1 a^{k+1} + C_2 \\ &\le (C_1 a) a^{\log_b n} + C_2 \\ &= (C_1 a) n^{\log_b a} + C_2, \end{aligned}$$

because $k \le \log_b n < k + 1$.

Hence, we have $f(n)$ is $O(n^{\log_b a})$.

Complexity of Binary Search

Binary Search Example: Give a big-*O* estimate for the number of comparisons used by a binary search.

Solution: Since the number of comparisons used by binary search is $f(n) = f(n/2) + 2$ where n is even, by Theorem 1, it follows that $f(n)$ is $O(\log n)$.

Example

EXAMPLE 6 Let $f(n) = 5f(n/2) + 3$ and $f(1) = 7$. Find $f(2^k)$, where k is a positive integer. Also, estimate $f(n)$ if f is an increasing function.

Extra Examples

Solution: From the proof of Theorem 1, with $a = 5$, $b = 2$, and $c = 3$, we see that if $n = 2^k$, then

$$\begin{aligned} f(n) &= a^k[f(1) + c/(a-1)] + [-c/(a-1)] \\ &= 5^k[7 + (3/4)] - 3/4 \\ &= 5^k(31/4) - 3/4. \end{aligned}$$

Also, if $f(n)$ is increasing, Theorem 1 shows that $f(n)$ is $O(n^{\log_b a}) = O(n^{\log 5})$. ◀

Estimating the Size of Divide-and-Conquer Functions

Theorem 2. Master Theorem: Let *f* be an increasing function that satisfies the recurrence relation

$$f(n) = af(n/b) + cn^d$$

whenever $n = b^k$, where k is a positive integer greater than 1, and c and d are real numbers with c positive and d nonnegative. Then

$$f(n) \text{ is } \begin{cases} O(n^d) & \text{if } a < b^d, \\ O(n^d \log n) & \text{if } a = b^d, \\ O(n^{\log_b a}) & \text{if } a > b^d. \end{cases}$$

Merge sort

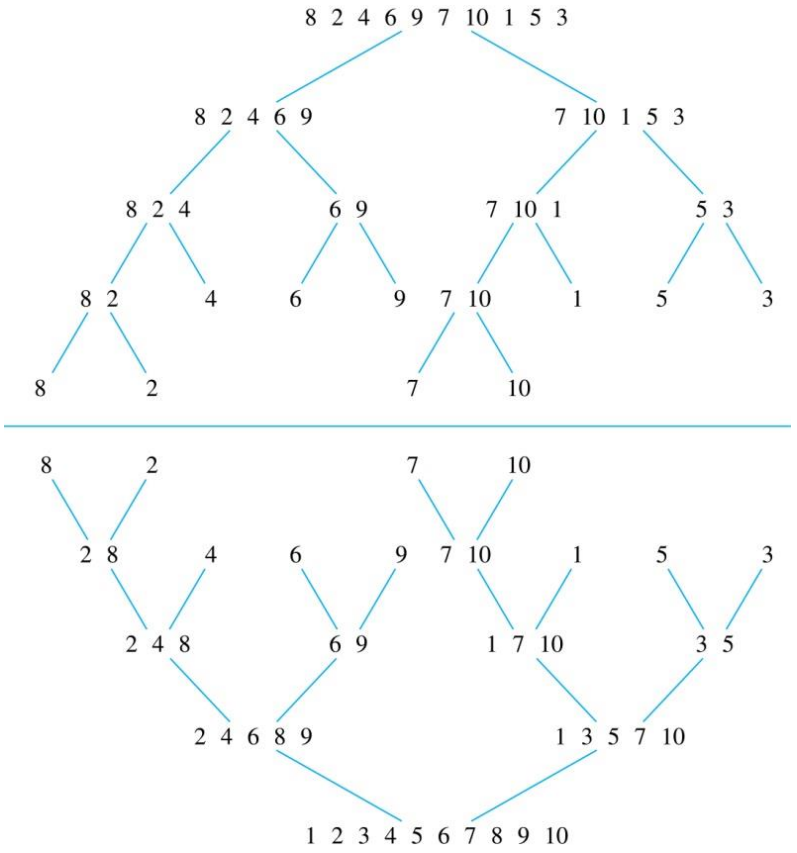
- *Merge Sort* works by iteratively splitting a list (with an even number of elements) into two sublists of equal length until each sublist has one element.
- Each sublist is represented by a balanced binary tree.
- At each step a pair of sublists is successively merged into a list with the elements in increasing order. The process ends when all the sublists have been merged.
- The succession of merged lists is represented by a binary tree.

Example

Use merge sort to put the list

8,2,4,6,9,7,10, 1, 5, 3

into increasing order.



Merge sort

- **Algorithm: a recursive merge sort**

  Procedure *mergesort*($L=a_1, \ldots, a_n$)

  if n>1 then

  m:=$\lfloor n/2 \rfloor$

  $L_1 := a_1, a_2, \ldots, a_m$

  $L_2 := a_{m+1}, a_{m+2}, \ldots, a_n$

  $L :=$ ***merge***(***mergesort***(L_1),***mergesort***(L_2))

  {*L* is now sorted into elements in nondecreasing order}

Merging two lists

- **Procedure merge(L_1, L_2)**

  L:=empty list

  while L_1 and L_2 are both empty

  begin

  remove smaller of first element of L_1 and L_2 from the list it is in and put it at the right end of L

  if removal of this element makes one list empty then remove all elements from the other list and append them to L

  end

  {L is the merged list with elements in increasing order}

Complexity of Merge Sort

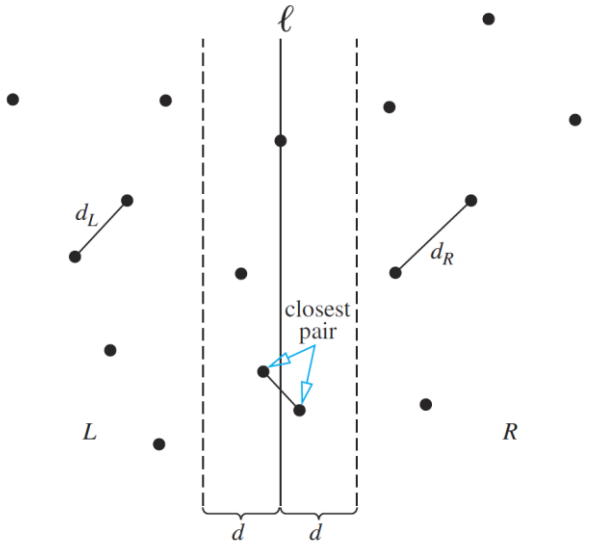
Merge Sort Example: Give a big-*O* estimate for the number of comparisons used by merge sort.

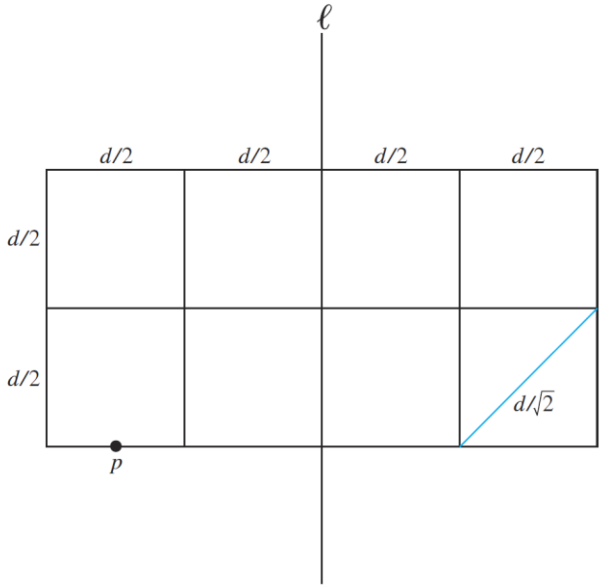
Solution: Since the number of comparisons used by merge sort to sort a list of n elements is less than $M(n)$ where $M(n) = 2M(n/2) + n$, by the master theorem $M(n)$ is $O(n \log n)$.

The Closest-Pair Problem

- **Page 559-561** ***O*(*n* log *n*)**

The Closest-Pair Problem Consider the problem of determining the closest pair of points in a set of n points $(x_1, y_1), \ldots, (x_n, y_n)$ in the plane, where the distance between two points (x_i, y_i) and (x_j, y_j) is the usual Euclidean distance $\sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$. This problem arises in many applications such as determining the closest pair of airplanes in the air space at a particular altitude being managed by an air traffic controller. How can this closest pair of points be found in an efficient way?





Exercises

- **Page 561: 12, 13, 20, 24, 29-31（Theorem 2 证明）**

8.4 Generating Functions

- Generating functions: are used to represent sequences efficiently by coding the terms of a sequence as coefficients of powers of a variety in a formal power series.
- Generating functions can be used to solve many types of counting problems.
- And solve recurrence relations by translating a recurrence relation for the terms of a sequence into an equation involving a generating function.
- prove combinatorial identities by taking advantages of relatively simple relationships between functions that can be translated into identities involving the terms of sequences.

Definition

- The generating function for the sequence a_0 , a_1, … , a_k … of real numbers is the infinite series

$$G(x) = a_0 + a_1 x + \cdots + a_k x^k + \cdots = \sum_{k=0}^{\infty} a_k x_k$$

Sequences	generating functions
3,3,3,…,3,…	$\sum_0^\infty 3x^k = 3/(1-x)$
1,2,3,…k+1,…	$\sum_0^\infty (k+1)x^k = 1/(1-x)^2$
1,2,4,…,2k,...	$\sum_0^\infty 2^k x^k = 1/(1-2x)$
1,1,1,1,1	$1+x+x^2+x^3+x^4+x^5$
c(m,0),c(m,1),…,c(m,m)	$(1+x)^m$

Useful facts about power series

- *Formal power series*: questions about their convergence are ignored.
- Important facts about infinite series used when working with generating functions. (calculus texts)

Useful facts about power series

- Let $f(x)=\sum_0^\infty a_k x^k$ and $g(x)=\sum_0^\infty b_k x^k$. Then

$$f(x)+g(x)=\sum_{k=0}^{\infty}(a_k+b_k)x^k$$

$$f(x)\bullet g(x)=\sum_{k=0}^{\infty}(\sum_{j=0}^{k}a_j b_{k-j})x^k$$

$$\alpha\bullet f(x)=\sum_{k=0}^{\infty}\alpha a_k x^k$$

$$x\bullet f'(x)=\sum_{k=0}^{\infty}k a_k x^k$$

$$f(\alpha x)=\sum_{k=0}^{\infty}\alpha^k a_k x^k$$

Examples

- The function $f(x)=1/(1-x)$ is the generating function of the sequence 1,1,1,…, since

  $1/(1-x)=1+x+x^2+\ldots$

- $1/(1-x)^2 \rightarrow 1,2,3,\ldots,k+1,\ldots$, since

$$1/(1-x)^2=\sum_{k=0}^{\infty}(\sum_{j=0}^{k}1)x^k=\sum_{k=0}^{\infty}(k+1)x^k$$

- $1/(1-x)^3 \rightarrow a_k=1+2+\ldots+k+1=(k+1)(k+2)/2$

- $f(x)=1/(1-4x^2) \rightarrow$?

  $1/(1-ax)=1+ax+a^2x^2+\ldots$

Extended binomial coefficient

- Definition: Let u be a real number and k a nonnegative integer. Then the extended binomial coefficient is defined by

$$\binom{u}{k} = \begin{cases} u(u-1)\cdots(u-k+1)/k! & \text{if } k > 0, \\ 1 & \text{if } k = 0. \end{cases}$$

- The extended binomial theorem: Let x be a real number with $|x|<1$ and let u be a real number. Then

$$(1+x)^u = \sum_{k=0}^{\infty} \binom{u}{k} x^k$$

Examples

- *C*(-1.5，3)=-1.5(-1.5 - 1)(-1.5 - 2) / 3! = -35/16
- When *u* = —*n* is a negative integer, the extended binomial coefficient can be expressed in terms of an ordinary binomial coefficient.

$$\binom{-n}{r} = \frac{(-n)(-n-1)\cdots(-n-r+1)}{r!} = \frac{(-1)^r (n+r-1)(n+r-2)\cdots n}{r!}$$

$$= \frac{(-1)^r (n+r-1)!}{r!(n-1)!} = (-1)^r \binom{n+r-1}{r} = (-1)^r C(n+r-1, r)$$

$$(1+x)^{-n} = \sum_{k=0}^{\infty} \binom{-n}{k} x^k = \sum_{k=0}^{\infty} (-1)^k C(n+k-1, k) x^k$$

$$(1-x)^{-n} = \sum_{k=0}^{\infty} C(n+k-1, k) x^k$$

Useful Generating Functions

- Page 568: table 1

Table 1 Useful Generating Functions.

$G(x)$	a_k
$(1+x)^n = \sum_{k=0}^{n} C(n,k)x^k$	$C(n,k)$
$(1+\alpha x)^n = \sum_{k=0}^{n} C(n,k)\alpha^k x^k$	$C(n,k)\alpha^k$
$(1+x^r)^n$	$C(n,k/r)$ if $r\|k$; 0 otherwise
$\frac{1-x^{n+1}}{1-x} = \sum_{k=0}^{n} x^k$	1 if $k \le n$; 0 otherwise
$\frac{1}{1-x} = \sum_{k=0}^{\infty} x^k$	1
$\frac{1}{1-\alpha x} = \sum_{k=0}^{\infty} \alpha^k x^k$	α^k
$\frac{1}{1-x^r} = \sum_{k=0}^{\infty} x^{rk}$	1 if $r\|k$; 0 otherwise
$\frac{1}{(1-x)^2} = \sum_{k=0}^{\infty}(k+1)x^k$	$k+1$
$\frac{1}{(1-x)^n} = \sum_{k=0}^{\infty} C(n+k-1,k)x^k$	$C(n+k+1,k)$
$\frac{1}{(1+x)^n} = \sum_{k=0}^{\infty} C(n+k-1,k)(-x)^k$	$(-1)^k C(n+k+1,k)$
$e^x = \sum_{k=0}^{\infty} \frac{x^k}{k!}$	$1/k!$

Counting problems and generating functions

- Generating functions can be used to solve a wide variety of counting problems.
  - Generating Functions and Combination

    1) Using generating functions to find the number of *k*-combination of a set with *n* elements.

    2) Using generating functions to find the number of *r*-combination from a set with *n* elements when the repetition of elements is allowed.
  - Generating Functions and Permutation

    The exponential generating function for the sequence $\{a_n\}$ is the series

$$\sum_{n=0}^{\infty} \frac{a_n}{n!} x^n$$

Generating Functions and Combination [1]

- Using generating functions to find the number of *k*-combination of a set with *n* elements.
- Consider the Binomial Theorem:

$$(1 + x)^n = (1 + x) \ldots (1 + x)$$
$$= C(n,0) + C(n,1)x + C(n,2)x^2 + \ldots + C(n,n)x^n$$

- Each of the *n* elements in the set contribute the term (1+ *x*) to the generating function

$$f(x) = \sum_{k=0}^{n} a_k x^k$$

Here $f(x)$ is the generating function for $\{a_k\}$, where a_k represents the number of *k*-combinations of a set with *n* elements.

Generating Functions and Combination [2]

- Using generating functions to find the number of *r*-combination from a set with *n* elements when the repetition of elements is allowed.
- Let $G(x) = \sum_0^\infty a_r x^r$ be the generating function for the sequence $\{a_r\}$, where a_r equals the number of *r*-combination from a set with *n* elements when the repetition of elements is allowed. Since each element contributes $(1+x+x^2+x^3+\dots)$ to a product expansion for $G(x)$, we have

  $$G(x) = (1 + x + x^2 + x^3 + \dots)^n$$

- As long as $|x| < 1$, we have $1 + x + x^2 + x^3 + \dots = 1/(1-x)$, so

  $$G(x) = 1/(1-x)^n = (1-x)^{-n}$$

- Applying the extended binomial theorem, it follows that

$$(1-x)^{-n} = (1+(-x))^{-n} = \sum_{r=0}^{\infty} \binom{-n}{r} (-x)^r$$

Generating Functions and Combination [3]

- The number of *r*-combination from a set with *n* elements with repetitions allowed, when *r* a positive integer, is the coefficient a_r of x^r in this sum. Consequently, we find that

$$a_r = \binom{-n}{r}(-1)^r = (-1)^r C(n+r-1, r)(-1)^r = C(n+r-1, r)$$

Examples [1]

- Find the number of the solutions of $e_1+e_2+e_3 = 17$, Where e_1,e_2,e_3 are nonnegative integers with

  $2 \le e_1 \le 5,\ 3 \le e_2 \le 6,\ 4 \le e_3 \le 7$

  The solutions is the coefficient of x^{17} in the expansion of

  $(x^2 + x^3 + x^4 + x^5)(x^3 + x^4 + x^5 + x^6)(x^4 + x^5 + x^6 + x^7) = \ldots + 3x^{17} + \ldots$

- Find the number of the solutions of $e_1+e_2+e_3 = 17$, Where e_1,e_2,e_3 are nonnegative odd integers

  The solutions is the coefficient of x^{17} in the expansion of

  $(x+x^3 + \ldots. + x^{2n-1} + \ldots.)^3 = x^3/(1- x^2)^3$

  $= \sum_0^{\infty} c(n+2,2)\, x^{2n+3}$

  Hence the coefficient of x^{17} is $c(9,2)=36$ is the number of solution

Examples [2]

- Use generating functions to determine the number of ways to insert tokens worth \$1, \$2, and \$5 into a vending machine to pay for an item that costs *r* dollars in both the cases when the order in which the tokens are inserted does not matter and when the order does matter. (For example, there are two ways to pay for an item that costs \$3 when the order in which the tokens are inserted does not matter: inserting three \$1 tokens or one \$1 token and a \$2 token. When the order matters, there are three ways: inserting three \$1 tokens, inserting a \$1 token and then a \$2 token, or inserting a \$2 token and then a \$1 token.)

$$(1 + x + x^2 + x^3 + \cdots)(1 + x^2 + x^4 + x^6 + \cdots)(1 + x^5 + x^{10} + x^{15} + \cdots).$$

$$1 + (x + x^2 + x^5) + (x + x^2 + x^5)^2 + \cdots = \frac{1}{1 - (x + x^2 + x^5)}$$

Examples [3]

- Use generating functions to find the number of *k*-combinations of a set with *n* elements. Assume that the binomial theorem has already been established.
- Use generating functions to find the number of *r*-combinations from a set with *n* elements when repetition of elements is allowed.
- Use generating functions to find the number of ways to select *r* objects of *n* different kinds if we must select at least one object of each kind.

*Generating Functions and Permutation

- Since $C(n; r) = P(n,r)=r!$, we need the following generating functions: The exponential generating function for the sequence $\{a_n\}$ is the series

$$\sum_{n=0}^{\infty} \frac{a_n}{n!} x^n$$

Examples [1]

- How many different strings with four characters can be formed from the letter in *CHANCE AND CHOICE*?

  There are {4*C*, 2*H*, 2*N*, 2*A*, 2*E*, *D*, *O*, *I* } in *CHANCE AND CHOICE*.

  The generating function to this problem is

$$G(x) = (1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \frac{x^4}{4!})(1 + x + \frac{x^2}{2!})^4 (1 + x)^4$$

  We need the coefficient $x^4/4!$. The coefficient of $x^4/4!$ in this product is 3029.

Examples [2]

- How many different strings with length *r* made from the digits 1, 3, 5, 7, and 9 are there that contain even 3s and even 7s?

  The generating function to this problem is

$$G(x) = (1 + \frac{x^2}{2!} + \frac{x^4}{4!} + \cdots)^2 (1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \frac{x^4}{4!} + \cdots)^3$$

Examples [3]

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \frac{x^4}{4!} + \cdots$$

and

$$e^{-x} = 1 - x + \frac{x^2}{2!} - \frac{x^3}{3!} + \frac{x^4}{4!} - \cdots$$

$$\begin{aligned} G(x) &= (\frac{e^x + e^{-x}}{2})^2 \cdot (e^x)^3 = \frac{1}{4}(e^{2x} + 2 + e^{-2x})e^{3x} \\ &= \frac{1}{4}(e^{5x} + 2e^{3x} + e^x) \\ &= \frac{1}{4}[\sum_{r=0}^{\infty} 5^r \frac{x^r}{r!} + 2\sum_{r=0}^{\infty} 3^r \frac{x^r}{r!} + \sum_{r=0}^{\infty} \frac{x^r}{r!}] \\ &= \sum_{r=0}^{\infty} (\frac{1}{4} \cdot 5^r + \frac{1}{2} \cdot 3^r + \frac{1}{4}) \frac{x^r}{r!} \end{aligned} \tag{5}$$

Therefore, there are $\frac{1}{4} \cdot 5^r + \frac{1}{2} \cdot 3^r + \frac{1}{4}$ strings with length r made from the digits 1, 3, 5, 7, and 9 are there that contain even 3s and even 7s.

Using generating functions to solving recurrence relations

- Find the solution to a recurrence relation and its initial conditions by finding an explicit formula for the associated generating function.
- Example: Solve the recurrence relation $a_k = 3a_{k-1}$ for $k = 1, 2, 3, \ldots$ and initial condition $a_0 = 2$.

Let $G(X)$ be the generating function for the sequence $\{a_k\}$,

that is, $G(x) = \sum_{k=0}^{\infty} a_k x^k$. First note that

$$xG(X) = \sum_{k=0}^{\infty} a_k x^{k+1} = \sum_{k=1}^{\infty} a_{k-1} x^k$$

Using the recurrence relation, we see that

$$G(X) - 3xG(X) = \sum_{k=0}^{\infty} a_k x^k - 3\sum_{k=1}^{\infty} a_{k-1} x^k$$

$$= a_0 + \sum_{k=1}^{\infty} (a_k - 3a_{k-1}) x^k$$

$$= 2$$

Example

- Continue:

Since $a_0 = 2$ and $a_k = 3a_{k-1}$. Thus,

$$G(x) - 3xG(x) = (1-3x)G(x) = 2.$$

Solving for $G(x)$ shows that $G(x) = 2/(1-3x)$. Using the identity $1/(1-ax) = \sum_{k=0}^{\infty} a^k x^k$, we have

$$G(x) = 2\sum_{k=0}^{\infty} 3^k x^k = \sum_{k=0}^{\infty} 2 \bullet 3^k x^k.$$

COnsequently, $a_k = 2 \cdot 3^k$

Example

- Suppose that a valid code word is an n-digit number in decimal notation containing an even numbers of 0s. Let a_n denote the number of valid code words of length n, then the sequence $\{a_n\}$ satisfies the recurrence relation

$$a_n = 8a_{n-1} + 10^{n-1}$$

and the initial condition $a_1 = 9$. Using generating functions to find an explicit formula for a_n.

Solution

We multiply both sides of the recurrence relation by x^n to obtain

$$a_n x^n = 8a_{n-1}x^n + 10^{n-1}x^n.$$

Let $G(x) = \sum_{n=0}^{\infty} a_n x^n$ be the generating function of the sequence $a_0, a_2, \cdots$.

We sum both sides of the last equation starting with $n = 1$, to find that

$$\begin{aligned} G(x) - 1 &= \sum_{n=1}^{\infty} a_n x^n = \sum_{n=1}^{\infty} (8a_{n-1}x^n + 10^{n-1}x^n) \\ &= 8\sum_{n=1}^{\infty} a_{n-1}x^n + \sum_{n=1}^{\infty} 10^{n-1}x^n \\ &= 8x\sum_{n=1}^{\infty} a_{n-1}x^{n-1} + x\sum_{n=1}^{\infty} 10^{n-1}x^{n-1} \\ &= 8x\sum_{n=0}^{\infty} a_n x^n + x\sum_{n=0}^{\infty} 10^n x^n \\ &= 8xG(x) + x/(1-10x). \end{aligned}$$

Solution

- Continue

Therefore, we have

$$G(x)-1=8xG(x)+x/(1-10x)$$

Solving for $G(x)$ shows that

$$G(x)=\frac{1-9x}{(1-8x)(1-10x)}$$

$$=\frac{1}{2}\left(\frac{1}{1-8x}+\frac{1}{1-10x}\right)$$

$$=\frac{1}{2}\left(\sum_{n=0}^{\infty}8^n x^n+\sum_{n=0}^{\infty}10^n x^n\right)$$

$$=\sum_{n=0}^{\infty}\frac{1}{2}(8^n+10^n)x^n$$

Consequently, we have shown that

$$a_n=\frac{1}{2}(8^n+10^n)$$

Using generating functions to prove identities

Use generating functions to show that

$$\sum_{k=0}^{n} C(n,k)^2 = C(2n,n)$$

whenever n is a positive integer.

Solution: First note that by the binomial theorem $C(2n,n)$ is the coefficient of x^n in $(1+x)^{2n}$. However, we also have

$$\begin{aligned}(1+x)^{2n} &= [(1+x)^n]^2 \\ &= [C(n,0) + C(n,1)x + C(n,2)x^2 + \cdots + C(n,n)x^n]^2.\end{aligned}$$

The coefficient of x^n in this expression is

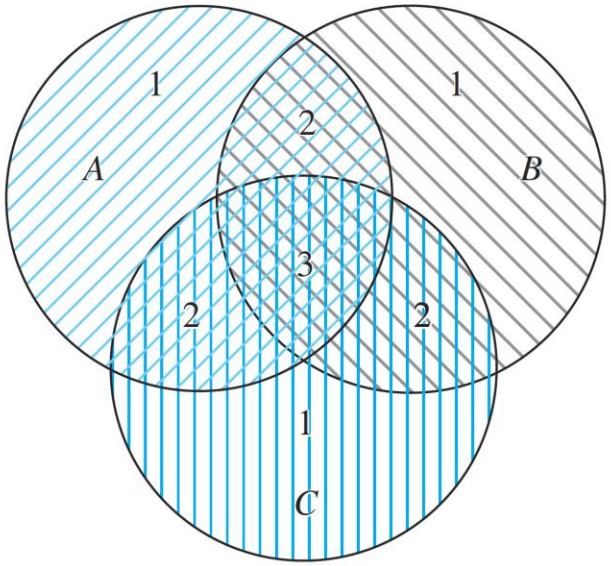
$$C(n,0)C(n,n) + C(n,1)C(n,n-1) + C(n,2)C(n,n-2) + \cdots + C(n,n)C(n,0).$$

This equals $\sum_{k=0}^{n} C(n,k)^2$, because $C(n,n-k) = C(n,k)$. Because both $C(2n,n)$ and $\sum_{k=0}^{n} C(n,k)^2$ represent the coefficient of x^n in $(1+x)^{2n}$, they must be equal. ◀

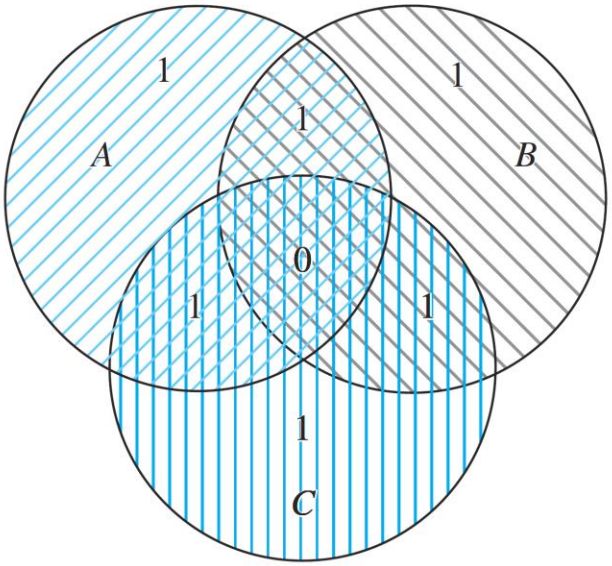
Exercises

- **Page 575: 11, 17, 25, 27, 35, 38, 44**

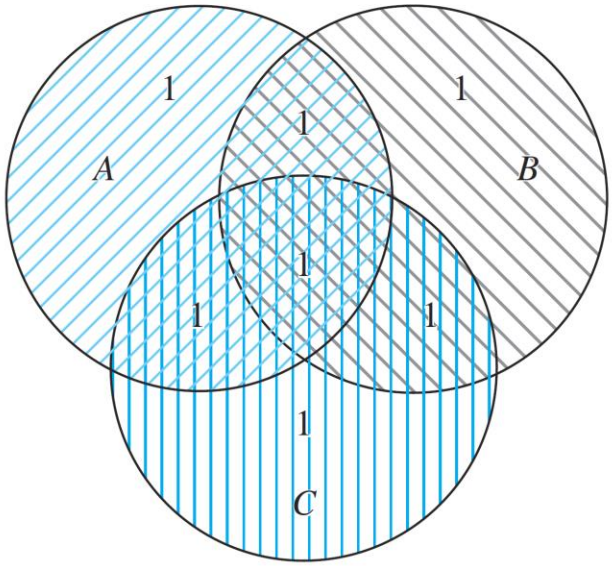
8.5 Inclusion-Exclusion



(a) Count of elements by $|A| + |B| + |C|$



(b) Count of elements by $|A| + |B| + |C| - |A \cap B| - |A \cap C| - |B \cap C|$



(c) Count of elements by $|A| + |B| + |C| - |A \cap B| - |A \cap C| - |B \cap C| + |A \cap B \cap C|$

8.5 Inclusion-Exclusion

- The principle of inclusion-exclusion: Let A_1, A_1,…, A_n be finite sets. Then

$$\left|A_1 \cup A_2 \cup \cdots \cup A_n\right| = \sum_{1\le i\le n}\left|A_i\right| - \sum_{1\le i<j\le n}\left|A_i \cap A_j\right|$$

$$+ \sum_{1\le i<j<k\le n}\left|A_i \cap A_j \cap A_k\right| - \cdots + (-1)^{n+1}\left|A_1 \cap A_2 \cap \cdots \cap A_n\right|$$

- Proof:
- Suppose that *a* is a member of exactly *r* of the sets A_1, A_1,…, A_n
- $C(r, 1) - C(r, 2) + C(r, 3) - \cdots + (-1)^{r+1} C(r, r)$

$$C(r, 0) - C(r, 1) + C(r, 2) - \cdots + (-1)^r C(r, r) = 0.$$

$$1 = C(r, 0) = C(r, 1) - C(r, 2) + \cdots + (-1)^{r+1} C(r, r).$$

Examples

- A total of 1232 students have taken a course in Spanish, 879 have taken a course in French, and 114 have taken a course in Russian. Further, 103 have taken courses in both Spanish and French, 23 have taken courses in both Spanish and Russian, and 14 have taken courses in both French and Russian. If 2092 students have taken a course in at least one of Spanish French and Russian, how many students have taken a course in all 3 languages.
- Solution: Let S be the set of students who have taken a course in Spanish, F the set of students who have taken a course in French, and R the set of students who have taken a course in Russian. Then, we have
- $|S| = 1232$, $|F| = 879$, $|R| = 114$, $|S \cap F| = 103$, $|S \cap R| = 23$, $|F \cap R| = 14$, and $|S \cup F \cup R| = 23$.
- Using the equation
- $|S \cup F \cup R| = |S| + |F| + |R| - |S \cap F| - |S \cap R| - |F \cap R| + |S \cap F \cap R|$,
- we obtain $2092 = 1232 + 879 + 114 - 103 - 23 - 14 + |S \cap F \cap R|$.
- Solving for $|S \cap F \cap R|$ yields 7.

Exercises

- **P. 584: 7, 13, 22**

8.6 Applications of Inclusion-Exclusion

- The Sieve of Eratosthenes
- Counting Onto-Functions
- Derangements

An alternative form of inclusion-exclusion

The number of elements in the set is N.

Let A_i be the subset containing the elements that have property P_i.

The number of elements with all the properties $P_{i1}, P_{i2}, \cdots, P_{ik}$ will be denoted by $N(P_{i1}P_{i2}\cdots P_{ik})$

The number of elements with none of the properties $P_1, P_2, \cdots, P_n$ is denoted by $N(P_1'P_2'\cdots P_n')$

$$N(P_1'P_2'\cdots P_n') = N - \sum_{1\le i\le n} N(P_i) + \sum_{1\le i<j\le n} N(P_iP_j)$$
$$- \sum_{1\le i<j<k\le n} N(P_iP_jP_k) + \cdots + (-1)^n N(P_1P_2\cdots P_n)$$

Example

How many solutions does

$$x_1 + x_2 + x_3 = 11$$

have, where x_1, x_2, and x_3 are nonnegative integers with $x_1 \leq 3$, $x_2 \leq 4$, and $x_3 \leq 6$?

Solution: To apply the principle of inclusion–exclusion, let a solution have property P_1 if $x_1 > 3$, property P_2 if $x_2 > 4$, and property P_3 if $x_3 > 6$. The number of solutions satisfying the inequalities $x_1 \leq 3$, $x_2 \leq 4$, and $x_3 \leq 6$ is

$$N(P_1' P_2' P_3') = N - N(P_1) - N(P_2) - N(P_3) + N(P_1 P_2) + N(P_1 P_3) + N(P_2 P_3) - N(P_1 P_2 P_3).$$

The sieve of Eratosthenes

- The principle of inclusion-exclusion can be used to find the number of primes not exceeding a specified positive integer.
- Recall that a composite integer is divisible by a prime not exceeding its square root.
- Example: the number of primes not exceeding 100

  2,3,5,7

  $4+N(P_1'P_2'P_3'P_4')$
- The sieve of Eratosthenes: is used to find all primes not exceeding a specified positive integer.

The number of onto functions

Example: How many onto functions are there from a set with six elements to a set with three elements?

Solution: Suppose that the elements in the codomain are b_1, b_2, and b_3. Let P_1, P_2, and P_3 be the properties that b_1, b_2, and b_3 are not in the range of the function, respectively. The function is onto if none of the properties P_1, P_2, and P_3 hold.

By the inclusion-exclusion principle the number of onto functions from a set with six elements to a set with three elements is

$$N - [N(P_1) + N(P_2) + N(P_3)] + [N(P_1P_2) + N(P_1P_3) + N(P_2P_3)] - N(P_1P_2P_3)$$

- **Here the total number of functions from a set with six elements to one with three elements is $N = 3^6$.**
- **The number of functions that do not have in the range is $N(P_1) = 2^6$. Similarly, $N(P_2) = N(P_3) = 2^6$.**
- **Note that $N(P_1P_2) = N(P_1P_3) = N(P_2P_3) = 1$ and $N(P_1P_2P_3) = 0$.**

$$3^6 - 3 \cdot 2^6 + 3 = 729 - 192 + 3 = 540$$

The number of onto functions

Theorem 1: Let m and n be positive integers with $m \geq n$. Then there are

$$n^m - C(n,1)(n-1)^m + C(n,2)(n-2)^m - \cdots + (-1)^{n-1}C(n,n-1)\cdot 1^m$$

onto functions from a set with *m* elements to a set with *n* elements.

Proof follows from the principle of inclusion-exclusion (*see Exercise* 27).

See Stirling number in Chapter 6

Derangement

Definition: A *derangement* is a permutation of objects that leaves no object in the original position.

Example: The permutation of 21453 is a derangement of 12345 because no number is left in its original position. But 21543 is not a derangement of 12345, because 4 is in its original position.

Derangement

- The principle of inclusion-exclusion will be used to count the permutations of *n* objects that leave no objects in their original positions. (derangement)
- Theorem 2: The number of derangements of a set with *n* elements is

$$D_n = n!\left[1 - \frac{1}{1!} + \frac{1}{2!} - \frac{1}{3!} + \cdots + (-1)^n \frac{1}{n!}\right]$$

Example

The Hatcheck Problem: A new employee checks the hats of *n* people at restaurant, forgetting to put claim check numbers on the hats. When customers return for their hats, the checker gives them back hats chosen at random from the remaining hats. What is the probability that no one receives the correct hat.

Solution: The answer is the number of ways the hats can be arranged so that there is no hat in its original position divided by *n*!, the number of permutations of *n* hats.

Remark: It can be shown that the probability of a derangement approaches $1/e$ as n grows without bound.

$$\frac{D_n}{n!} = \left[1 - \frac{1}{1!} + \frac{1}{2!} - \frac{1}{3!} + \cdots + (-1)^n \frac{1}{n!}\right]$$

TABLE 1 The Probability of a Derangement.

n	2	3	4	5	6	7
$D_n/n!$	0.50000	0.33333	0.37500	0.36667	0.36806	0.36786

Exercises

- **P. 591: 7, 9, 11, 13, 17**